Multi-Cluster Scheduling of DAG-Based Tasks Using Priority Functions

Marisha Thorat
Undergraduate Project Report
Indian Institute of Technology Kanpur

Abstract

Efficient scheduling of dependent tasks across heterogeneous resources is a key challenge in modern High Performance Computing (HPC) systems. This project addresses this challenge by designing a multi-cluster scheduling strategy for tasks represented as Directed Acyclic Graphs (DAGs), capturing inter-task dependencies.

A priority-based scheduling algorithm is proposed that dynamically selects tasks using a weighted combination of critical path length, resource utilization, waiting time, and dependency constraints. The scheduler operates across multiple resource types, enabling effective allocation in heterogeneous environments such as CPU and GPU clusters.

The proposed approach is evaluated against baseline policies, including First-Come First-Serve (FCFS) with backfilling, across multiple test cases. Results demonstrate that dependency-aware prioritization improves scheduling efficiency, particularly for workloads with long dependency chains.

Experiments with different weight configurations further highlight the sensitivity of the algorithm to parameter tuning, emphasizing the importance of balancing multiple scheduling objectives.

1 Introduction

High Performance Computing (HPC) systems increasingly execute complex applications composed of multiple interdependent tasks. These tasks are naturally modeled as Directed Acyclic Graphs (DAGs), where nodes represent tasks and edges represent dependencies.

Efficient scheduling of such DAG-based workloads is challenging due to several factors, including dependency constraints between tasks, heterogeneous resource requirements (such as CPUs and GPUs), dynamic arrival of jobs, and the need to balance multiple performance objectives such as utilization, fairness, and makespan.

Traditional scheduling approaches often fail to account for these factors simultaneously, leading to suboptimal performance. This motivates the need for dependency-aware and resource-aware scheduling strategies tailored for multi-cluster environments.

2 Problem Statement

The objective of this work is to design an efficient scheduling strategy for tasks represented as Directed Acyclic Graphs (DAGs) in a multi-cluster computing environment.

In such systems, tasks exhibit dependency constraints and require heterogeneous resources, such as CPUs and GPUs, while also arriving dynamically over time. These characteristics make scheduling particularly challenging, as decisions must simultaneously account for dependency satisfaction, resource availability, and system-level performance goals.

The scheduler must therefore ensure that task dependencies are respected, effectively utilize heterogeneous resources, adapt to dynamic job arrivals, and optimize key performance metrics such as makespan and resource utilization.

3 Related Work

3.1 Baseline Scheduling: FCFS with Conservative Backfilling

For comparison, a First-Come First-Serve (FCFS) scheduling policy augmented with conservative backfilling is used as a baseline. In this approach, tasks are primarily considered in the order of their ready times (i.e., when all dependencies are satisfied), while respecting resource constraints.

Under FCFS, the earliest ready task (referred to as the head task) is given priority for execution. If the head task cannot be scheduled immediately due to insufficient resources, a reservation is made for its earliest possible start time based on currently running tasks.

Backfilling is then applied to improve resource utilization. Other ready tasks may be scheduled out of order, provided that their execution does not delay the reserved start time of the head task. This ensures fairness to earlier tasks while reducing idle resource time.

This conservative backfilling strategy does not incorporate task priority, critical path information, or global optimization metrics. Consequently, while it improves utilization compared to pure FCFS, it may still lead to suboptimal scheduling decisions in complex scenarios involving deep dependency structures and heterogeneous resources. Therefore, it serves as a meaningful and stronger baseline for evaluating the effectiveness of the proposed scheduling algorithm.

4 Methodology

4.1 Proposed Scheduling Algorithm

4.1.1 Overview

The proposed scheduler operates on tasks represented as nodes in a Directed Acyclic Graph (DAG), where edges denote dependencies between tasks. The algorithm dynamically selects tasks based on a priority function that incorporates multiple factors such as critical path length, resource utilization, waiting time, and dependency constraints.

4.1.2 Task Representation

```
struct Task {
    int id, app_id, cluster_type;
    double duration, resource_req;

    double CP;
    double remWork;

    double ready_time;
    double arrival_time;
    bool runnable;
    bool done;
    bool added;

    vector<int> children;
    vector<int> parents;
};
```

Each task maintains dependency information along with scheduling-related attributes. Tasks become runnable only after all parent tasks are completed.

4.1.3 Critical Path Computation

```
double computeCP(int u, vector<Task> &tasks, vector<double> &dp) {
    if (dp[u] != -1) return dp[u];

    double max_child = 0;
    for (int v : tasks[u].children) {
        max_child = max(max_child, computeCP(v, tasks, dp));
    }

    return dp[u] = tasks[u].duration + max_child;
}
```

The critical path represents the longest execution path from a task to the end of the DAG. It is computed using memoized recursion.

4.1.4 Priority Function

```
double computePriority(Task &t, double current_time, double fit,
   double depPenalty) {
    double waiting = current_time - t.ready_time;

    double w1 = 2, w2 = 0.5, w3 = 0.05, alpha = 0.02, w4 = 2;

    return w1 * t.CP
```

```
        + w2 * fit
        - w3 * t.remWork
        + alpha * waiting
        - w4 * depPenalty;
}
```

This function assigns a score to each runnable task based on multiple scheduling factors. The task with the highest score is selected for execution.

4.1.5 Task Selection Mechanism

```
int selectNextTask(vector<Task> &tasks, double current_time,
                   double available_resource, int cluster_type) {

    int best_task = -1;
    double best_score = -1e18;

    for (auto &t : tasks) {

        if (!t.added) continue;
        if (!t.runnable || t.done) continue;
        if (t.cluster_type != cluster_type) continue;

        double fit = t.resource_req / available_resource;

        double depPenalty = 0;
        for (int child : t.children) {
            for (int p : tasks[child].parents) {
                if (p != t.id && !tasks[p].done) {
                    depPenalty = max(depPenalty, tasks[p].duration);
                }
            }
        }

        double score = computePriority(t, current_time, fit,
           depPenalty);

        if (t.resource_req <= available_resource && score >
           best_score) {
            best_score = score;
            best_task = t.id;
        }
    }

    return best_task;
}
```

This function iterates over all eligible tasks and selects the one with the highest priority score, subject to resource constraints.

4.1.6 Task Completion and Dependency Handling

```
void completeTask(int id, vector<Task> &tasks, double current_time,
                  unordered_map<int, double> &remWork) {

    tasks[id].done = true;
    tasks[id].runnable = false;

    remWork[tasks[id].app_id] -= tasks[id].duration;

    for (int child : tasks[id].children) {

        bool ready = true;

        for (int p : tasks[child].parents) {
            if (!tasks[p].done) {
                ready = false;
                break;
            }
        }

        if (ready && !tasks[child].done) {
            tasks[child].runnable = true;
            tasks[child].ready_time = current_time;
        }
    }
}
```

Upon completion of a task, its children are checked. If all dependencies are satisfied, they are marked runnable.

4.2 Priority Formula (Key Contribution)

$$Priority = w_1 \cdot CP + w_2 \cdot fit - w_3 \cdot remWork + \alpha \cdot waiting - w_4 \cdot depPenalty$$

Interpretation of terms:

- CP: Prioritizes tasks on the critical path.
- fit: Encourages efficient resource utilization.
- $remWork$: Prevents bias toward applications with large remaining workloads.
- $waiting$: Improves fairness by reducing starvation.

- *depPenalty*: Penalizes tasks whose execution may delay dependent tasks.

The weights $(w_1, w_2, w_3, w_4, \alpha)$ control the relative importance of each factor and are varied during experimentation to study their impact on performance.

The task with the highest computed priority is selected for execution, subject to resource availability constraints.

5 Implementation

5.1 Initial SLURM Setup and Exploration

At the beginning of the project, an effort was made to understand and set up the SLURM workload manager. This involved configuring and running key SLURM daemons such as `slurmctld` and `slurmd`, along with experimenting with job submission and execution. This phase helped build familiarity with the SLURM environment and informed the subsequent design of the scheduling approach.

5.2 Main Scheduling Loop

The scheduler operates in simulated time, progressing through discrete events. At each step:

- Newly arrived tasks are added.
- Runnable tasks are identified.
- Tasks are scheduled on CPU and GPU resources separately.
- The system advances to the next event (task completion or arrival).

5.3 Input Format for Test Cases

Each test case consists of a set of tasks representing multiple jobs (DAGs). The input begins with an integer n denoting the total number of tasks across all jobs. This is followed by n lines, each describing a single task.

Each task is represented using the following format:

```
id   app_id   cluster_type   duration   resource_req   arrival_time   k
                        parent_1   parent_2   ...
```

The parameters are defined as follows:

- **id**: Unique identifier of the task.
- **app_id**: Identifier of the job (DAG) to which the task belongs.
- **cluster_type**: Type of resource required for execution (e.g., CPU or GPU).
- **duration**: Execution time of the task.

- **resource_req**: Amount of resource units required by the task.
- **arrival_time**: Time at which the job (DAG) becomes available in the system. All tasks belonging to the same job share the same arrival time.
- **k**: Number of parent tasks (dependencies).
- **parent_i**: Identifiers of the parent tasks that must complete before this task can start.

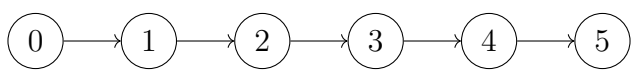
A task is considered *runnable* only when all its parent tasks have completed execution and its job has arrived in the system. This representation allows modeling of arbitrary Directed Acyclic Graphs (DAGs) with varying structures and resource requirements.

6 Experiments

6.1 DAG Types

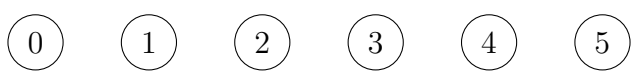
To systematically evaluate scheduling performance, we define several types of Directed Acyclic Graphs (DAGs), each representing a different structural pattern of task dependencies. These types are designed to stress different aspects of scheduling such as dependency depth, parallelism, and synchronization.

6.1.1 Type 1: Linear Chain



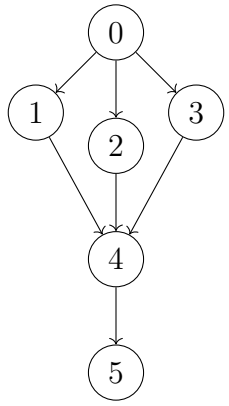
This structure represents a purely sequential workflow with no parallelism. It is used to evaluate dependency handling and serves as a baseline scenario.

6.1.2 Type 2: Fully Parallel



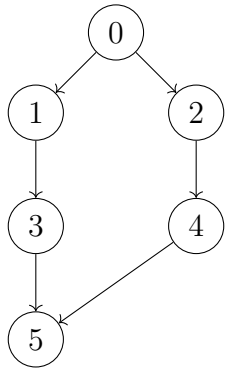
All tasks are independent and can execute concurrently. This type tests resource allocation and scheduling efficiency.

6.1.3 Type 3: Fork-Join



This structure introduces synchronization points where multiple parallel branches must complete before proceeding. It evaluates how well the scheduler handles dependency convergence.

6.1.4 Type 4: Diamond Structure



This structure contains multiple paths that merge at later stages. It is useful for analyzing critical path prioritization and scheduling decisions across competing paths.

6.1.5 Type 5: Heterogeneous (Heavy-Light Mix)

This type consists of tasks with varying execution times and resource requirements, arranged with partial dependencies. It is designed to evaluate backfilling behavior and the scheduler's ability to utilize resources efficiently under heterogeneous workloads.

6.2 Experimental Design

To evaluate the effectiveness of the proposed scheduling algorithm, a diverse set of test cases was designed by systematically varying key workload characteristics.

6.2.1 Design Factors

Each test case is constructed by varying the following dimensions:

- **DAG Structure:** Linear, Fully Parallel, Fork-Join, Diamond, and Mixed graphs.
- **Arrival Pattern:** Simultaneous, staggered, bursty, and delayed arrivals.
- **Task Heterogeneity:** Variation in execution time and resource requirements (heavy vs light tasks).
- **Cluster Type:** CPU and GPU assignments, including skewed and irregular distributions.

6.2.2 Test Case Overview

- **TC1:** Linear chains (baseline correctness)
- **TC2:** Fully parallel DAGs (resource utilization)
- **TC3:** Fork-Join DAGs (synchronization behavior)
- **TC4:** Diamond DAGs with staggered arrivals (critical path sensitivity)
- **TC5:** Heavy-light mixed DAGs (backfilling behavior)
- **TC6:** Mixed DAG structures (realistic workload)
- **TC7:** Parallel DAGs with tight stagger (dynamic contention)
- **TC8:** Fork-Join DAGs with irregular cluster usage (heterogeneous dependencies)
- **TC9:** Mixed DAGs with bursty arrivals (congestion handling)
- **TC10:** Heavy early jobs with late small jobs (fairness vs prioritization)

This design enables evaluation across a wide range of realistic scheduling scenarios.

6.3 Weight Configurations

To evaluate the impact of different scheduling objectives, two distinct weight configurations are used in the priority function. Each configuration emphasizes a different aspect of scheduling, enabling a comparative analysis of trade-offs between throughput and fairness.

6.3.1 Configuration 1: Critical Path Oriented

- $w_1 = 2.5$ (Critical Path)
- $w_2 = 0.5$ (Resource Fit)
- $w_3 = 0.05$ (Remaining Work)
- $\alpha = 0.02$ (Waiting Time)
- $w_4 = 1.0$ (Dependency Penalty)

This configuration prioritizes tasks on longer critical paths, aiming to minimize overall makespan. It focuses on completing deep dependency chains early, potentially at the cost of fairness and increased waiting time for smaller tasks.

6.3.2 Configuration 2: Fairness and Responsiveness Oriented

- $w_1 = 1.0$ (Critical Path)
- $w_2 = 0.4$ (Resource Fit)
- $w_3 = 0.2$ (Remaining Work)
- $\alpha = 0.08$ (Waiting Time)
- $w_4 = 1.0$ (Dependency Penalty)

This configuration emphasizes fairness by prioritizing tasks from smaller jobs and those that have waited longer. It reduces starvation and improves responsiveness, though it may lead to slightly higher makespan compared to the critical-path-focused configuration.

These two configurations allow us to study the trade-off between throughput-oriented and fairness-oriented scheduling strategies.

7 Results

The proposed scheduler achieves an average improvement of approximately 7–8% over the FCFS baseline (excluding the synchronization-heavy case TC4), with peak improvements exceeding 15% in bursty workload scenarios.

Test Case	Weight 1	Weight 2	FCFS + Backfill
TC1	67	68	75
TC2	87	87	89
TC3	86	86	92
TC4	89	80	69
TC5	89	89	97
TC6	116	114	118
TC7	115	115	118
TC8	95	95	101
TC9	87	86	103
TC10	142	143	151

Table 1: Comparison of makespan across different scheduling strategies

Test Case	W1 Improvement (%)	W2 Improvement (%)
TC1	10.67	9.33
TC2	2.25	2.25
TC3	6.52	6.52
TC4	-28.99	-15.94
TC5	8.25	8.25
TC6	1.69	3.39
TC7	2.54	2.54
TC8	5.94	5.94
TC9	15.53	16.50
TC10	5.96	5.30

Table 2: Percentage improvement over FCFS baseline

8 Performance Visualization

To better illustrate the comparative performance of the scheduling strategies, the results are visualized using graphs.

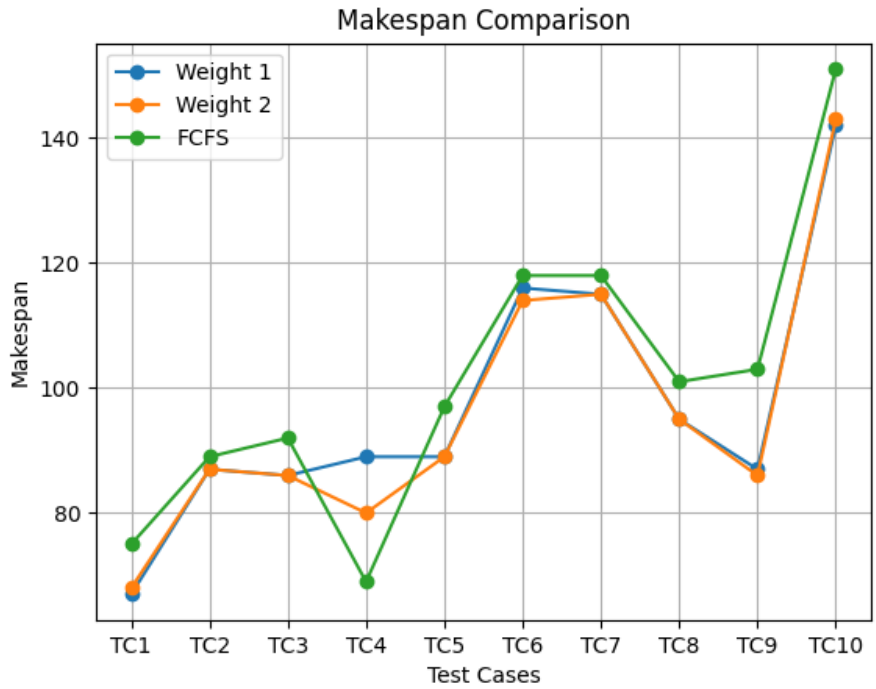


Figure 1: Comparison of makespan across test cases

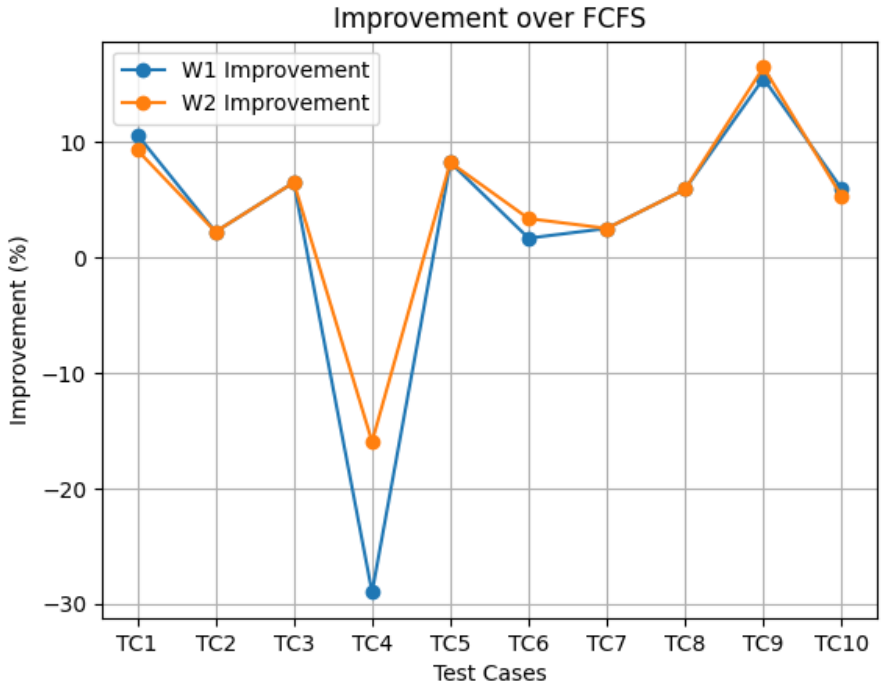


Figure 2: Percentage improvement over FCFS baseline

9 Conclusions

9.1 Performance Analysis

9.1.1 Overall Improvement

Overall, the scheduler achieves an average improvement of approximately 7–8%, with peak gains exceeding 15% in challenging scenarios, demonstrating its effectiveness for multi-cluster scheduling of DAG-based workloads.

9.1.2 Behavior Across Workload Types

- **Simple Workloads (TC1–TC3):** Minimal improvements due to limited scheduling flexibility.
- **Heterogeneous Workloads (TC5, TC6):** Moderate improvements due to better resource utilization.
- **Dynamic Workloads (TC7–TC9):** Significant improvements as the scheduler adapts to arrival patterns.
- **Synchronization-heavy Workloads (TC4):** Performance degradation due to imbalance across dependency branches.

9.1.3 Impact of Cluster Heterogeneity

The scheduler performs particularly well in scenarios with mixed CPU and GPU workloads, effectively balancing resource utilization and reducing idle time.

9.2 Detailed Analysis of Test Case 4

Test Case 4 consists of diamond-shaped DAGs, which introduce synchronization points where multiple parallel branches must complete before execution can proceed. The execution logs reveal a clear difference in scheduling behavior between FCFS and the proposed priority-based scheduler.

Under FCFS with backfilling, tasks belonging to the same DAG are executed in a relatively grouped manner. This results in balanced progress across parallel branches. Consequently, both branches leading to a synchronization point complete around the same time, allowing dependent tasks to begin without significant delay. This behavior leads to a lower makespan of 69.

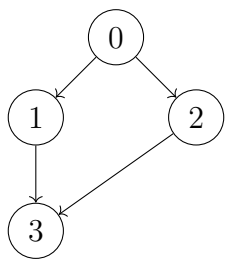
In contrast, the proposed scheduler exhibits significant interleaving across different DAGs. Tasks from multiple jobs are scheduled based on priority scores, leading to scattered execution. For example, tasks from unrelated DAGs are introduced early in the schedule, interrupting the progression of parallel branches within the same DAG.

This results in branch imbalance, where one branch of a diamond structure completes earlier while the other is delayed. Since the join node requires all parent tasks to complete, this imbalance introduces waiting periods that increase the overall makespan.

The issue is more pronounced in the critical-path-focused configuration (Weight 1), which aggressively prioritizes high-impact tasks across different DAGs. This leads to maximum interleaving and the highest makespan of 89. The fairness-oriented configuration (Weight 2) performs slightly better (makespan 80) due to reduced starvation, but still suffers from lack of structural balance.

These observations highlight a key limitation of priority-based scheduling: while it improves global task selection, it does not account for the need to maintain synchronized progress across parallel branches within a DAG. In synchronization-heavy structures such as diamond graphs, preserving this balance is more important than optimizing individual task priorities.

Illustrative Example Consider a simple diamond-shaped DAG:



If the scheduler prioritizes task 1 and delays task 2 due to interleaving with other jobs, task 3 cannot begin even though one branch has already completed. This creates idle waiting at the synchronization point, increasing the overall completion time.

In contrast, a more balanced execution where tasks 1 and 2 progress simultaneously allows task 3 to start earlier, reducing makespan.

9.3 Analysis of Test Case 9

Test Case 9 consists of mixed DAGs with bursty arrivals, where multiple jobs enter the system simultaneously after an initial phase. This creates sudden spikes in system load and significant contention for resources.

The execution logs reveal that the FCFS with backfilling approach performs poorly in this scenario, with a makespan of 103. Since FCFS primarily follows arrival order, early jobs dominate execution, causing later-arriving jobs to experience significant delays. Tasks from the burst phase (e.g., tasks with higher IDs) are scheduled much later, indicating starvation and poor adaptability to dynamic workload conditions.

In contrast, the proposed scheduler effectively interleaves tasks from different jobs. Tasks from the burst phase are scheduled much earlier, as seen in the execution logs where higher-index tasks appear significantly sooner compared to FCFS. This behavior is driven by the waiting-time component in the priority function, which increases the priority of newly arrived tasks, and the critical-path component, which prioritizes structurally important tasks.

Unlike synchronization-heavy cases, interleaving across jobs is beneficial in this scenario. It improves resource utilization, reduces queue buildup, and prevents starvation. The scheduler dynamically adapts to workload changes, leading to a significant reduction in makespan (86–87).

This test case highlights the strength of the proposed scheduling approach in handling dynamic and bursty workloads, where flexibility and fairness play a crucial role in achieving efficient execution.

9.4 Weight Configuration Analysis

Two weight configurations were evaluated to study the trade-off between critical-path prioritization and fairness.

9.4.1 Comparison of Configurations

Overall, the fairness-oriented configuration (Weight 2) performs slightly better than the critical-path-focused configuration (Weight 1), particularly in dynamic and bursty workloads.

9.4.2 Observations

- In static scenarios, both configurations perform similarly.
- In dynamic environments, the fairness component helps reduce waiting time and improves responsiveness.
- In synchronization-heavy workloads, both configurations exhibit similar behavior due to structural constraints.

These results indicate that while critical-path awareness is important, incorporating fairness improves robustness in realistic workloads.

9.5 Final Conclusion

The proposed scheduling approach demonstrates consistent improvements over the FCFS baseline, particularly in dynamic and heterogeneous environments. While gains are modest in simple scenarios, significant improvements are observed under bursty and resource-constrained conditions.

The study highlights the importance of balancing critical-path prioritization with fairness to achieve robust performance across diverse workloads.

Overall, the scheduler achieves an average improvement of approximately 7–8%, with peak gains exceeding 15% in challenging scenarios, validating its effectiveness for multi-cluster scheduling.